1 Getting started with Spring

This chapter covers

- Spring and Spring Boot essentials
- Initializing a Spring project
- An overview of the Spring landscape

Although the Greek philosopher Heraclitus wasn't well known as a software developer, he seems to have had a good handle on the subject. He has been quoted as saying, "The only constant is change." That statement captures a foundational truth of software development.

The way we develop applications today is different than it was a year ago, 5 years ago, 10 years ago, and certainly 20 years ago, before an initial form of the Spring Framework was introduced in Rod Johnson's book, *Expert One-on-One J2EE Design and Development* (Wrox, 2002, **http://mng.bz/oVjy**).

Back then, the most common types of applications developed were browser-based web applications, backed by relational databases. Although that type of development is still relevant—and Spring is well equipped for those kinds of applications—we're now also interested in developing applications composed of microservices destined for the cloud that persist data in a variety of databases. And a new interest in reactive programming aims to provide greater scalability and improved performance with nonblocking operations.

As software development evolved, the Spring Framework also changed to address modern development concerns, including microservices and reactive programming. The creators of Spring also set out to simplify its development model by introducing Spring Boot.

Whether you're developing a simple database-backed web application or constructing a modern application built around microservices, Spring is the framework that will help you achieve your goals. This chapter is your first step in a journey through modern application development with Spring.

1.1 What is Spring?

I know you're probably itching to start writing a Spring application, and I assure you that before this chapter ends, you'll have developed a simple one. But first, let me set the stage with a few basic Spring concepts that will help you understand what makes Spring tick.

Any nontrivial application comprises many components, each responsible for its own piece of the overall application functionality, coordinating with the other application elements to get the job done. When the application is run, those components somehow need to be created and introduced to each other.

At its core, Spring offers a *container*, often referred to as the *Spring application context*, that creates and manages application components. These components, or *beans*, are wired together inside the Spring application context to make a complete application, much like bricks, mortar, timber, nails, plumbing, and wiring are bound together to make a house.

The act of wiring beans together is based on a pattern known as *dependency injection* (DI). Rather than have components create and maintain the life cycle of other beans that they depend on, a dependency-injected application relies on a separate entity (the container) to create and maintain all components and inject those into the beans that need them. This is done typically through constructor arguments or property accessor methods.

For example, suppose that among an application's many components, you will address two: an inventory service (for fetching inventory levels) and a product service (for providing basic product information). The product service depends on the inventory service to be able to provide a complete set of information about products. Figure 1.1 illustrates the relationships between these beans and the Spring application context.

On top of its core container, Spring and a full portfolio of related libraries offer a web framework, a variety of data persistence options, a security framework, integration with other systems, runtime monitoring, microservice support, a reactive programming model, and many other features necessary for modern application development.

Historically, the way you would guide Spring's application context to wire beans together was with one or more XML files that described the components and their relationship to other components.

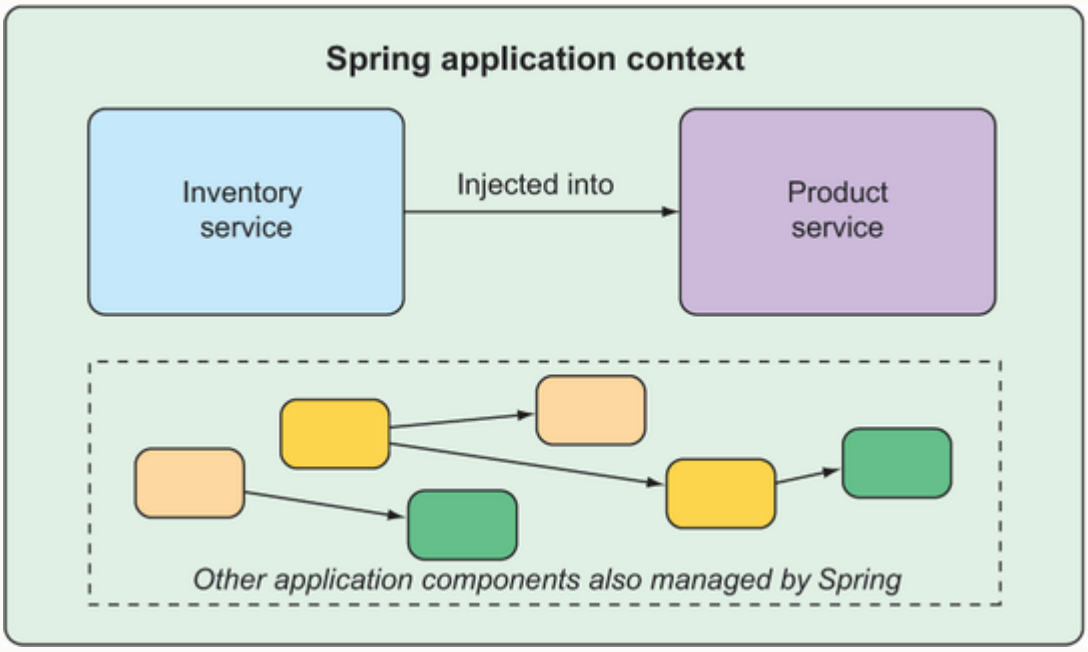


Figure 1.1 Application components are managed and injected into each other by the Spring application context.

For example, the following XML code declares two beans, an `InventoryService` bean and a `ProductService` bean, and wires the `InventoryService` bean into `ProductService` via a constructor argument:

```
<bean id="inventoryService"
    class="com.example.InventoryService" />

<bean id="productService"
    class="com.example.ProductService" >
 <constructor-arg ref="inventoryService" />
</bean>
```

In recent versions of Spring, however, a Java-based configuration is more common. The following Java-based configuration class is equivalent to the XML configuration:

```
@Configuration
public class ServiceConfiguration {
 @Bean
 public InventoryService inventoryService() {
  return new InventoryService();
 }

 @Bean
 public ProductService productService() {
  return new ProductService(inventoryService());
```

```
  }
}
```

The `@Configuration` annotation indicates to Spring that this is a configuration class that will provide beans to the Spring application context.

The configuration's methods are annotated with `@Bean`, indicating that the objects they return should be added as beans in the application context (where, by default, their respective bean IDs will be the same as the names of the methods that define them).

Java-based configuration offers several benefits over XML-based configuration, including greater type safety and improved refactorability. Even so, explicit configuration with either Java or XML is necessary only if Spring is unable to automatically configure the components.

Automatic configuration has its roots in the Spring techniques known as *autowiring* and *component scanning*. With component scanning, Spring can automatically discover components from an application's classpath and create them as beans in the Spring application context. With autowiring, Spring automatically injects the components with the other beans that they depend on.

More recently, with the introduction of Spring Boot, automatic configuration has gone well beyond component scanning and autowiring. Spring Boot is an extension of the Spring Framework that offers several productivity enhancements. The most well known of these enhancements is *autoconfiguration*, where Spring Boot can make reasonable guesses at what components need to be configured and wired together, based on entries in the classpath, environment variables, and other factors.

I'd like to show you some example code that demonstrates autoconfiguration, but I can't. Autoconfiguration is much like the wind—you can see the effects of it, but there's no code that I can show you and say "Look! Here's an example of autoconfiguration!" Stuff happens, components are enabled, and functionality is provided without writing code. It's this lack of code that's essential to autoconfiguration and what makes it so wonderful.

Spring Boot autoconfiguration has dramatically reduced the amount of explicit configuration (whether with XML or Java) required to build an application. In fact, by the time you finish the example in this chapter, you'll have a working Spring application that has only a single line of Spring configuration code!

Spring Boot enhances Spring development so much that it's hard to imagine developing Spring applications without it. For that reason, this book treats Spring and Spring Boot as if they were one and the same. We'll use Spring Boot as much as possible and explicit configuration only when necessary. And, because Spring XML configuration is the old-school way of working with Spring, we'll focus primarily on Spring's Java-based configuration.

But enough of this chitchat, yakety-yak, and flimflam. This book's title includes the phrase *in action*, so let's get moving, so you can start writing your first application with Spring.

1.2 Initializing a Spring application

Through the course of this book, you'll create Taco Cloud, an online application for ordering the most wonderful food created by man—tacos. Of course, you'll use Spring, Spring Boot, and a variety of related libraries and frameworks to achieve this goal.

You'll find several options for initializing a Spring application. Although I could walk you through the steps of manually creating a project directory structure and defining a build specification, that's wasted time—time better spent writing application code. Therefore, you're going to lean on the Spring Initializr to bootstrap your application.

The Spring Initializr is both a browser-based web application and a REST API, which can produce a skeleton Spring project structure that you can flesh out with whatever functionality you want. Several ways to use Spring Initializr follow:

- From the web application at **http://start.spring.io**
- From the command line using the `curl` command
- From the command line using the Spring Boot command-line interface
- When creating a new project with Spring Tool Suite
- When creating a new project with IntelliJ IDEA
- When creating a new project with Apache NetBeans

Rather than spend several pages of this chapter talking about each one of these options, I've collected those details in the appendix. In this chapter, and throughout this book, I'll show you how to create a new project using my favorite option: Spring Initializr support in Spring Tool Suite.

As its name suggests, Spring Tool Suite is a fantastic Spring development environment that comes in the form of extensions for Eclipse, Visual Studio Code, or the Theia IDE. You can download ready-to-run binaries of Spring Tool Suite at **https://spring.io/tools**.

Spring Tool Suite offers a handy Spring Boot Dashboard feature that makes it easy to start, restart, and stop Spring Boot applications from the IDE.

If you're not a Spring Tool Suite user, that's fine; we can still be friends. Hop over to the appendix and substitute the Initializr option that suits you best for the instructions in the following sections. But know that throughout this book, I may occasionally reference features specific to Spring Tool Suite, such as the Spring Boot Dashboard. If you're not using Spring Tool Suite, you'll need to adapt those instructions to fit your IDE.

2.1 Initializing a Spring project with Spring Tool Suite

To get started with a new Spring project in Spring Tool Suite, go to the File menu and select New, and then select Spring Starter Project. Figure 1.2 shows the menu structure to look for.

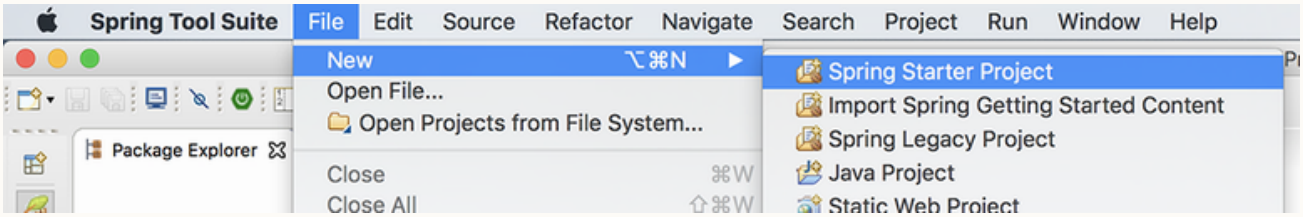


Figure 1.2 Starting a new project with the Initializr in Spring Tool Suite

Once you select Spring Starter Project, a new project wizard dialog (figure 1.3) appears. The first page in the wizard asks you for some general project information, such as the project name, description, and other essential information. If you're familiar with the contents of a Maven pom.xml file, you'll recognize most of the fields as items that end up in a Maven build specification. For the Taco Cloud application, fill in the dialog as shown in figure 1.3, and then click Next.

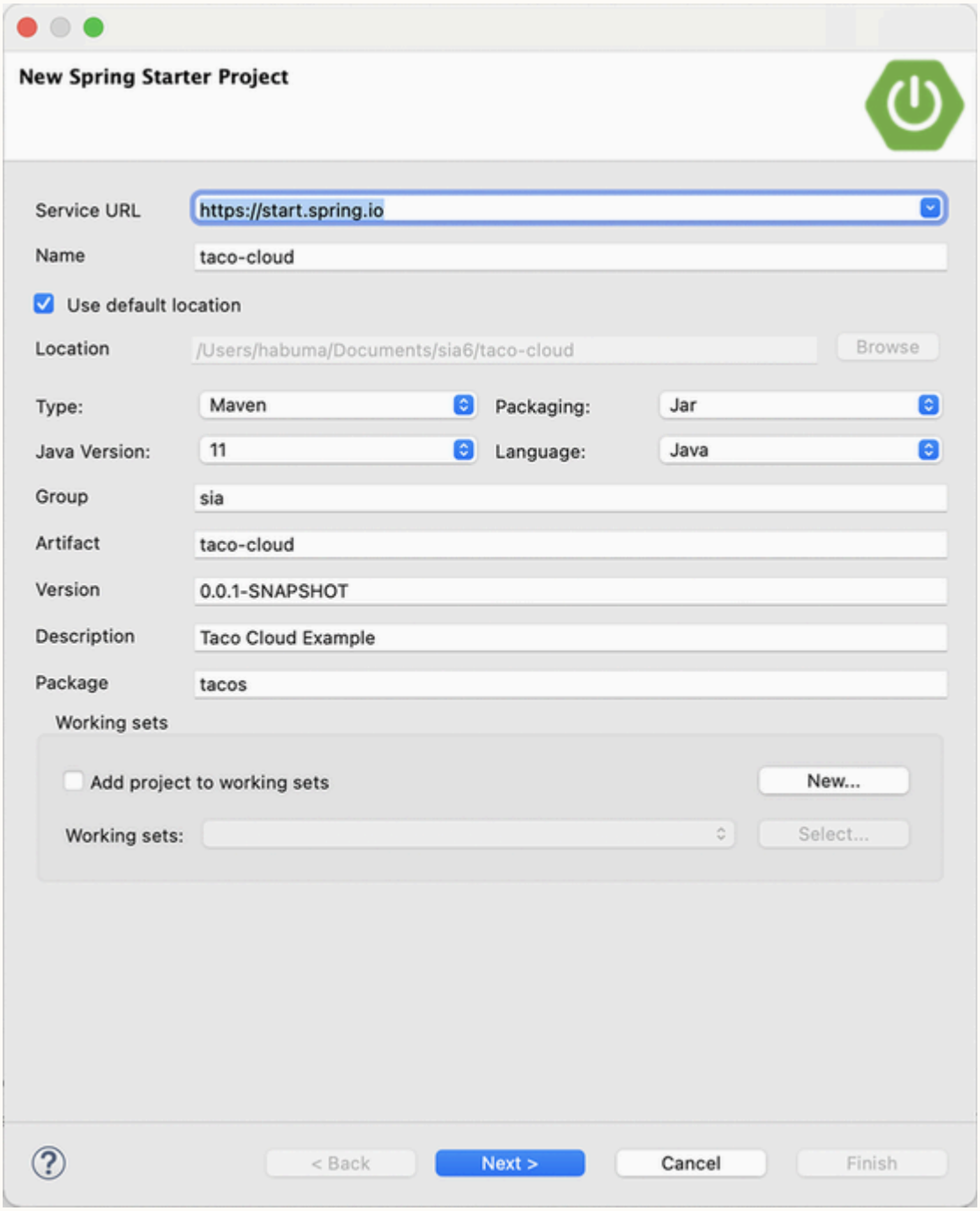


Figure 1.3 Specifying general project information for the Taco Cloud application

The next page in the wizard lets you select dependencies to add to your project (see figure 1.4). Notice that near the top of the dialog, you can select on which version of Spring Boot you want to base your project. This defaults to the most current version available. It's generally a good idea to leave it as is unless you need to target a different version.

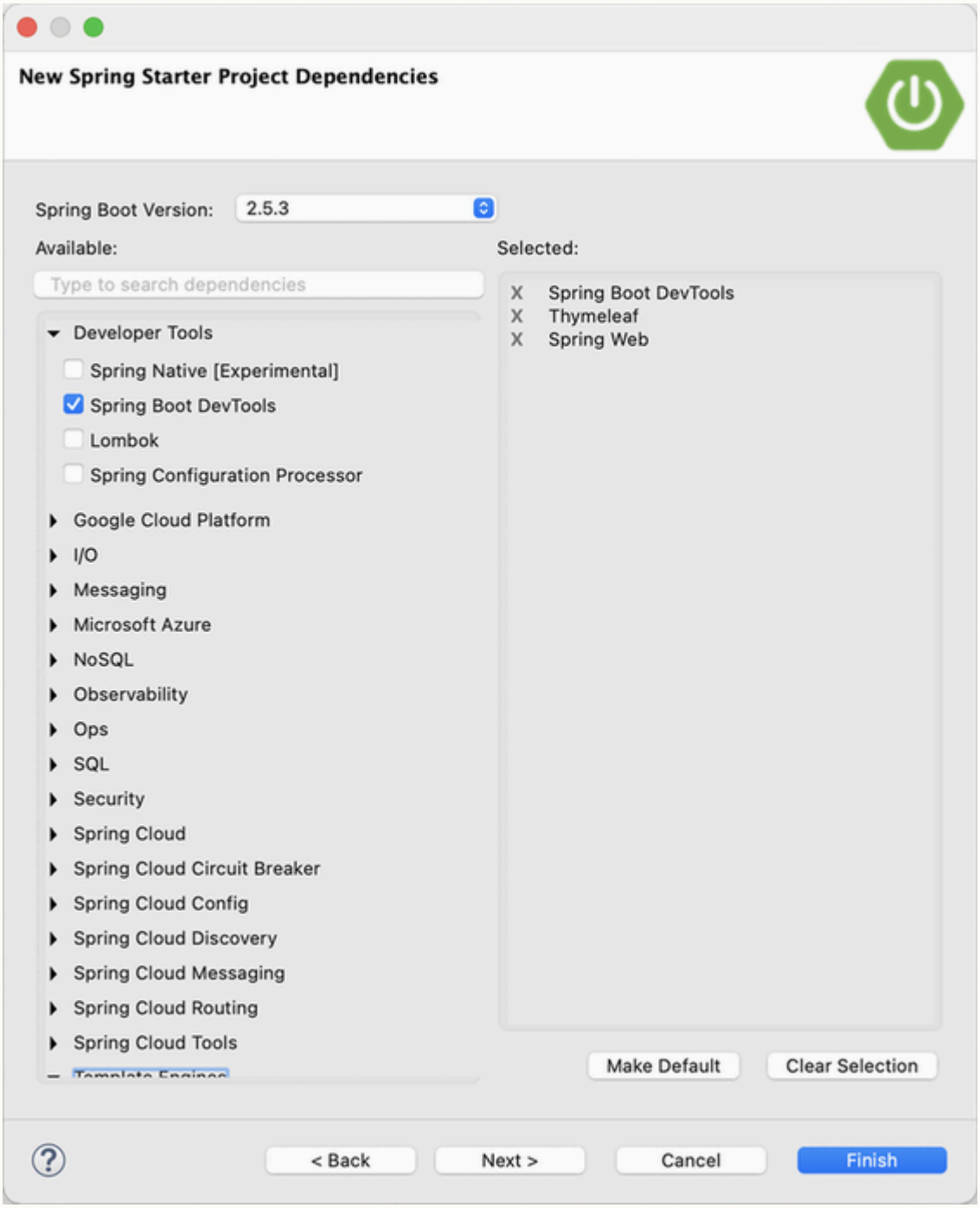


Figure 1.4 Choosing starter dependencies

As for the dependencies themselves, you can either expand the various sections and seek out the desired dependencies manually or search for them in the search box at the top of the Available list. For the Taco Cloud application, you'll start with the dependencies shown in figure 1.4.

At this point, you can click Finish to generate the project and add it to your workspace. But if you're feeling slightly adventurous, click Next one more time to see the final page of the new starter project wizard, as shown in figure 1.5.

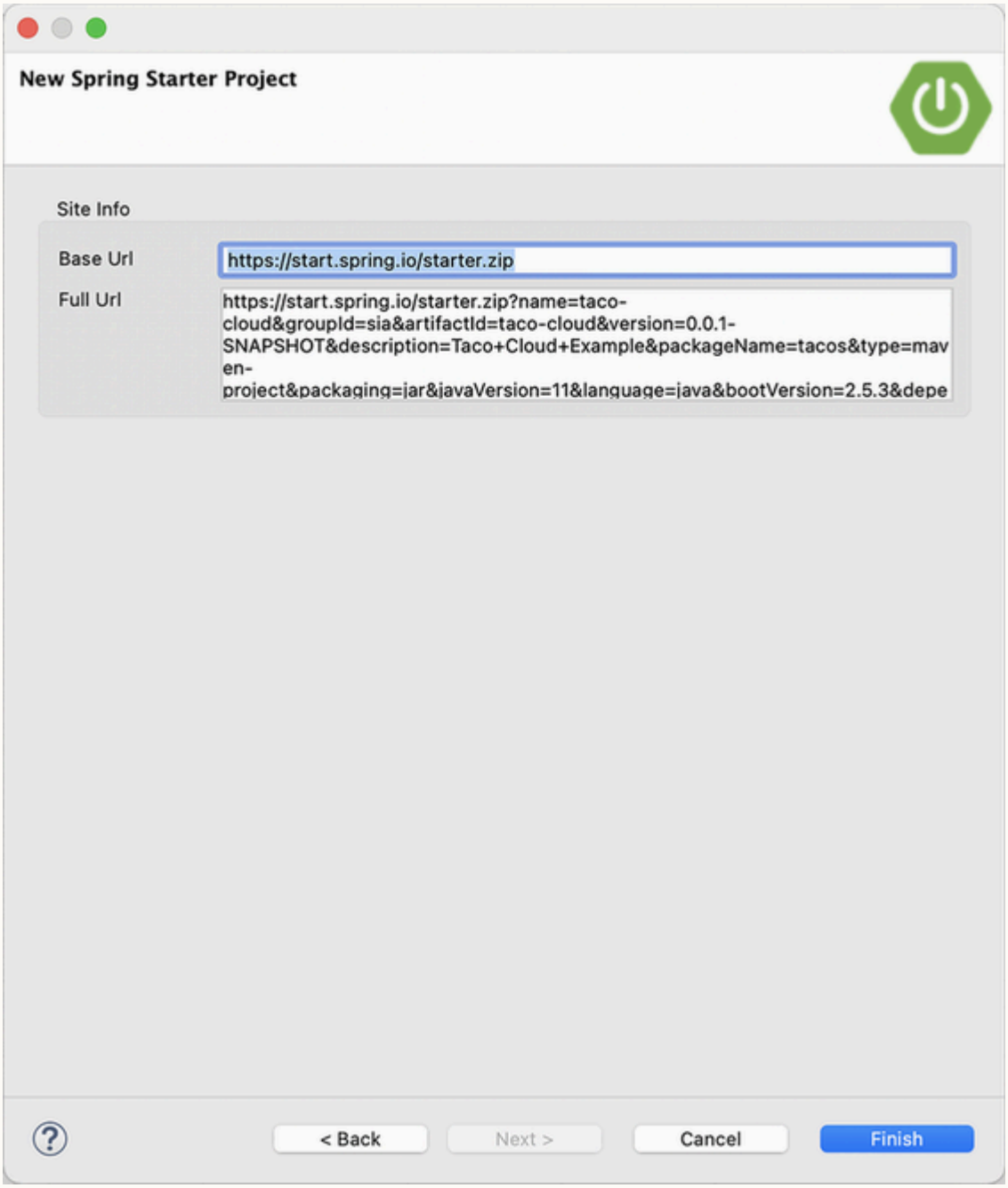


Figure 1.5 Optionally specifying an alternate Initializr address

By default, the new project wizard makes a call to the Spring Initializr at http://start.spring.io to generate the project. Generally, there's no need to override this default, which is why you could have clicked Finish on the second page of the wizard. But if for some reason you're hosting your own clone of Initializr (perhaps a local copy on your own machine or a customized clone running inside your company firewall), then you'll want to change the Base Url field to point to your Initializr instance before clicking Finish.

After you click Finish, the project is downloaded from the Initializr and loaded into your workspace. Wait a few moments for it to load and build, and then you'll be ready to start developing application functionality. But first, let's take a look at what the Initializr gave you.

2.2 Examining the Spring project structure

After the project loads in the IDE, expand it to see what it contains. Figure 1.6 shows the expanded Taco Cloud project in Spring Tool Suite.

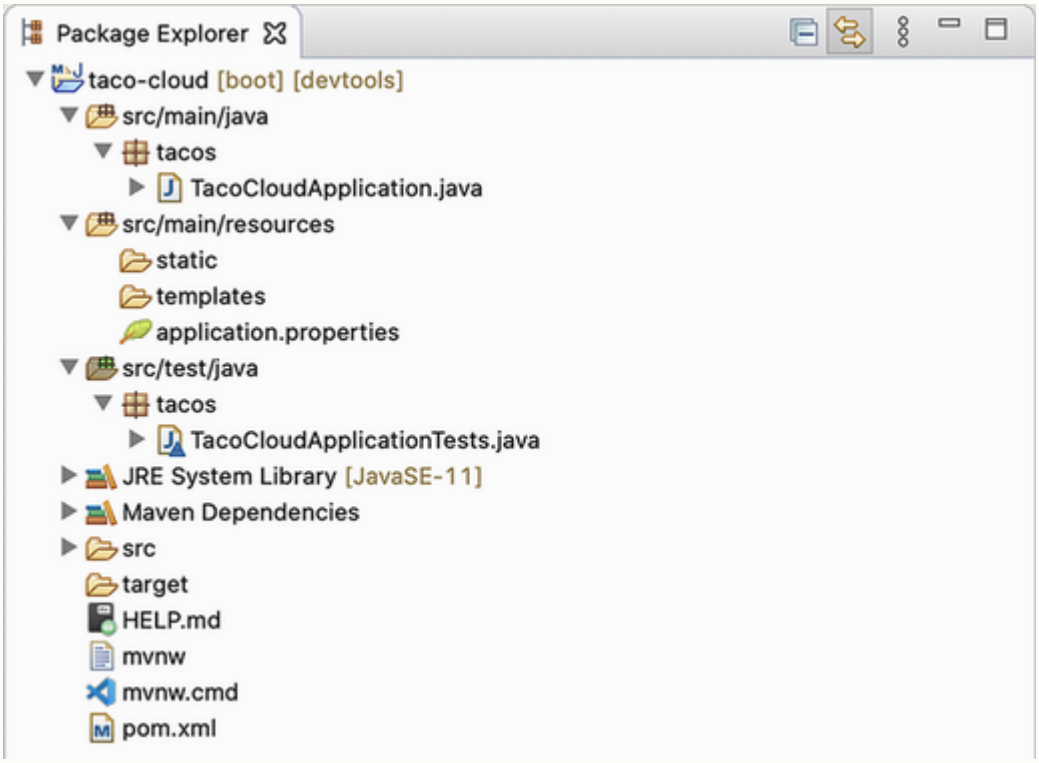


Figure 1.6 The initial Spring project structure as shown in Spring Tool Suite

You may recognize this as a typical Maven or Gradle project structure, where application source code is placed under src/main/java, test code is placed under src/test/java, and non-Java resources are placed under src/main/resources. Within that project structure, you'll want to take note of the following items:

- `mvnw` *and* `mvnw.cmd` —These are Maven wrapper scripts. You can use these scripts to build your project, even if you don't have Maven installed on your machine.
- *pom.xml*—This is the Maven build specification. We'll look deeper into this in a moment.
- `TacoCloudApplication.java` —This is the Spring Boot main class that bootstraps the project. We'll take a closer look at this class in a moment.
- *application.properties*—This file is initially empty but offers a place where you can specify configuration properties. We'll tinker with this file a little in this chapter, but I'll postpone a detailed explanation of configuration properties to chapter 6.
- *static*—This folder is where you can place any static content (images, stylesheets, JavaScript, and so forth) that you want to serve to the browser. It's initially empty.

- *templates*—This folder is where you'll place template files that will be used to render content to the browser. It's initially empty, but you'll add a Thymeleaf template soon.
- `TacoCloudApplicationTests.java`—This is a simple test class that ensures that the Spring application context loads successfully. You'll add more tests to the mix as you develop the application.

As the Taco Cloud application grows, you'll fill in this barebones project structure with Java code, images, stylesheets, tests, and other collateral that will make your project more complete. But in the meantime, let's dig a little deeper into a few of the items that Spring Initializr provided.

EXPLORING THE BUILD SPECIFICATION

When you filled out the Initializr form, you specified that your project should be built with Maven. Therefore, the Spring Initializr gave you a pom.xml file already populated with the choices you made. The following listing shows the entire pom.xml file provided by the Initializr.

Listing 1.1 The initial Maven build specification

```
<?xml version="1.0" encoding="UTF-8"?><project
xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
      https://maven.apache.org/xsd/maven-4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>2.5.3</version>                    (1)
  <relativePath />
 </parent>
 <groupId>sia</groupId>
 <artifactId>taco-cloud</artifactId>
 <version>0.0.1-SNAPSHOT</version>
 <name>taco-cloud</name>
 <description>Taco Cloud Example</description>

 <properties>
  <java.version>11</java.version>
 </properties>
```

```
<dependencies>
 <dependency>                          ❷
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
 </dependency>

 <dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
 </dependency>

 <dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
 </dependency>

 <dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
  <exclusions>
   <exclusion>
    <groupId>org.junit.vintage</groupId>
    <artifactId>junit-vintage-engine</artifactId>
   </exclusion>
  </exclusions>
 </dependency>

</dependencies>

<build>
 <plugins>
  <plugin>                             ❸
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-maven-plugin</artifactId>
  </plugin>
 </plugins>
</build>

<repositories>
 <repository>
  <id>spring-milestones</id>
  <name>Spring Milestones</name>
  <url>https://repo.spring.io/milestone</url>
```

```
    </repository>
  </repositories>
  <pluginRepositories>
    <pluginRepository>
      <id>spring-milestones</id>
      <name>Spring Milestones</name>
      <url>https://repo.spring.io/milestone</url>
    </pluginRepository>
  </pluginRepositories>

</project>
```

❶ Spring Boot version

❷ Starter dependencies

❸ Spring Boot plugin

The first thing to take note of is the `<parent>` element and, more specifically, its `<version>` child. This specifies that your project has `spring-boot-starter-parent` as its parent POM. Among other things, this parent POM provides dependency management for several libraries commonly used in Spring projects. For those libraries covered by the parent POM, you won't have to specify a version, because it's inherited from the parent. The version, `2.5.6`, indicates that you're using Spring Boot 2.5.6 and, thus, will inherit dependency management as defined by that version of Spring Boot. Among other things, Spring Boot's dependency management for version 2.5.6 specifies that the underlying version of the core Spring Framework will be 5.3.12.

While we're on the subject of dependencies, note that there are four dependencies declared under the `<dependencies>` element. The first three should look somewhat familiar to you. They correspond directly to the `Spring Web`, `Thymeleaf`, and `Spring Boot DevTools` dependencies that you selected before clicking the Finish button in the Spring Tool Suite new project wizard. The other dependency is one that provides a lot of helpful testing capabilities. You didn't have to check a box for it to be included because the Spring Initializr assumes (hopefully, correctly) that you'll be writing tests.

You may also notice that all dependencies except for the DevTools dependency have the word *starter* in their artifact ID. Spring Boot starter dependencies are special in that they typically don't have any library code themselves but instead transitively pull in other libraries. These starter dependencies offer the following primary benefits:

- Your build file will be significantly smaller and easier to manage because you won't need to declare a dependency on every library you might need.
- You're able to think of your dependencies in terms of what capabilities they provide, rather than their library names. If you're developing a web application, you'll add the web starter dependency rather than a laundry list of individual libraries that enable you to write a web application.
- You're freed from the burden of worrying about library versions. You can trust that the versions of the libraries brought in transitively will be compatible for a given version of Spring Boot. You need to worry only about which version of Spring Boot you're using.

Finally, the build specification ends with the Spring Boot plugin. This plugin performs a few important functions, described next:

- It provides a Maven goal that enables you to run the application using Maven.
- It ensures that all dependency libraries are included within the executable JAR file and available on the runtime classpath.
- It produces a manifest file in the JAR file that denotes the bootstrap class (`TacoCloudApplication`, in your case) as the main class for the executable JAR.

Speaking of the bootstrap class, let's open it up and take a closer look.

BOOTSTRAPPING THE APPLICATION

Because you'll be running the application from an executable JAR, it's important to have a main class that will be executed when that JAR file is run. You'll also need at least a minimal amount of Spring configuration to bootstrap the application. That's what you'll find in the `TacoCloudApplication` class, shown in the following listing.

Listing 1.2 The Taco Cloud bootstrap class

```
package tacos;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication                                    (1)
public class TacoCloudApplication {

  public static void main(String[] args) {
    SpringApplication.run(TacoCloudApplication.class, args);  (2)
  }
```

```
}
```

❶ Spring Boot application

❷ Runs the application

Although there's little code in `TacoCloudApplication`, what's there packs quite a punch. One of the most powerful lines of code is also one of the shortest. The `@SpringBootApplication` annotation clearly signifies that this is a Spring Boot application. But there's more to `@SpringBootApplication` than meets the eye.

`@SpringBootApplication` is a composite annotation that combines the following three annotations:

- `@SpringBootConfiguration`—Designates this class as a configuration class. Although there's not much configuration in the class yet, you can add Java-based Spring Framework configuration to this class if you need to. This annotation is, in fact, a specialized form of the `@Configuration` annotation.
- `@EnableAutoConfiguration`—Enables Spring Boot automatic configuration. We'll talk more about autoconfiguration later. For now, know that this annotation tells Spring Boot to automatically configure any components that it thinks you'll need.
- `@ComponentScan`—Enables component scanning. This lets you declare other classes with annotations like `@Component`, `@Controller`, and `@Service` to have Spring automatically discover and register them as components in the Spring application context.

The other important piece of `TacoCloudApplication` is the `main()` method. This is the method that will be run when the JAR file is executed. For the most part, this method is boilerplate code; every Spring Boot application you write will have a method similar or identical to this one (class name differences notwithstanding).

The `main()` method calls a static `run()` method on the SpringApplication class, which performs the actual bootstrapping of the application, creating the Spring application context. The two parameters passed to the `run()` method are a configuration class and the command-line arguments. Although it's not necessary that the configuration class passed to `run()` be the same as the bootstrap class, this is the most convenient and typical choice.

Chances are you won't need to change anything in the bootstrap class. For simple applications, you might find it convenient to configure one or two other components in the bootstrap class, but for most applications, you're better off creating a separate configuration class for anything that isn't autoconfigured. You'll define several configuration classes throughout the course of this book, so stay tuned for details.

TESTING THE APPLICATION

Testing is an important part of software development. You can always test your project manually by building it and then running it from the command line like this:

```
$ ./mvnw package
...
$ java -jar target/taco-cloud-0.0.1-SNAPSHOT.jar
```

Or, because we're using Spring Boot, the Spring Boot Maven plugin makes it even easier, as shown next:

```
$ ./mvnw spring-boot:run
```

But manual testing implies that there's a human involved and thus potential for human error and inconsistent testing. Automated tests are more consistent and repeatable.

Recognizing this, the Spring Initializr gives you a test class to get started. The following listing shows the baseline test class.

Listing 1.3 A baseline application test

```
package tacos;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest                    (1)
public class TacoCloudApplicationTests {

  @Test                            (2)
  public void contextLoads() {
  }
```

```
}
```

❶ A Spring Boot test

❷ The test method

There's not much to be seen in `TacoCloudApplicationTests`: the one test method in the class is empty. Even so, this test class does perform an essential check to ensure that the Spring application context can be loaded successfully. If you make any changes that prevent the Spring application context from being created, this test fails, and you can react by fixing the problem.

The `@SpringBootTest` annotation tells JUnit to bootstrap the test with Spring Boot capabilities. Just like `@SpringBootApplication`, `@SpringBootTest` is a composite annotation, which is itself annotated with `@ExtendWith(SpringExtension.class)`, to add Spring testing capabilities to JUnit 5. For now, though, it's enough to think of this as the test class equivalent of calling `SpringApplication.run()` in a `main()` method. Over the course of this book, you'll see `@SpringBootTest` several times, and we'll uncover some of its power.

Finally, there's the test method itself. Although `@SpringBootTest` is tasked with loading the Spring application context for the test, it won't have anything to do if there aren't any test methods. Even without any assertions or code of any kind, this empty test method will prompt the two annotations to do their job and load the Spring application context. If there are any problems in doing so, the test fails.

To run this and any test classes from the command line, you can use the following Maven incantation:

```
$ ./mvnw test
```

At this point, we've concluded our review of the code provided by the Spring Initializr. You've seen some of the boilerplate foundation that you can use to develop a Spring application, but you still haven't written a single line of code. Now it's time to fire up your IDE, dust off your keyboard, and add some custom code to the Taco Cloud application.

1.3 Writing a Spring application

Because you're just getting started, we'll start off with a relatively small change to the Taco Cloud application, but one that will demonstrate a lot of Spring's goodness. It seems appropriate that as you're just starting, the first feature you'll add to the Taco Cloud application is a home page. As you add the home page, you'll create the following two code artifacts:

- A controller class that handles requests for the home page
- A view template that defines what the home page looks like

And because testing is important, you'll also write a simple test class to test the home page. But first things first ... let's write that controller.

3.1 Handling web requests

Spring comes with a powerful web framework known as Spring MVC. At the center of Spring MVC is the concept of a *controller*, a class that handles requests and responds with information of some sort. In the case of a browser-facing application, a controller responds by optionally populating model data and passing the request on to a view to produce HTML that's returned to the browser.

You're going to learn a lot about Spring MVC in chapter 2. But for now, you'll write a simple controller class that handles requests for the root path (for example, `/`) and forwards those requests to the home page view without populating any model data. The following listing shows the simple controller class.

Listing 1.4 The home page controller

```
package tacos;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller                    (1)
public class HomeController {

  @GetMapping("/")             (2)
  public String home() {
    return "home";             (3)
  }
```

```
}
```

❶ The controller

❷ Handles requests for the root path /

❸ Returns the view name

As you can see, this class is annotated with `@Controller`. On its own, `@Controller` doesn't do much. Its primary purpose is to identify this class as a component for component scanning. Because `HomeController` is annotated with `@Controller`, Spring's component scanning automatically discovers it and creates an instance of `HomeController` as a bean in the Spring application context.

In fact, a handful of other annotations (including `@Component`, `@Service`, and `@Repository`) serve a purpose similar to `@Controller`. You could have just as effectively annotated `HomeController` with any of those other annotations, and it would have still worked the same. The choice of `@Controller` is, however, more descriptive of this component's role in the application.

The `home()` method is as simple as controller methods come. It's annotated with `@GetMapping` to indicate that if an HTTP `GET` request is received for the root path /, then this method should handle that request. It does so by doing nothing more than returning a `String` value of `home`.

This value is interpreted as the logical name of a view. How that view is implemented depends on a few factors, but because Thymeleaf is in your classpath, you can define that template with Thymeleaf.

Why Thymeleaf?

You may be wondering why I chose Thymeleaf for a template engine. Why not JSP? Why not FreeMarker? Why not one of several other options?

Put simply, I had to choose something, and I like Thymeleaf and generally prefer it over those other options. And even though JSP may seem like an obvious choice, there are some challenges to overcome when using JSP with Spring Boot. I didn't want to go down that rabbit hole in chapter 1. Hang tight. We'll look at other template options, including JSP, in chapter 2.

The template name is derived from the logical view name by prefixing it with /templates/ and postfixing it with .html. The resulting path for the template is /templates/home.html. Therefore, you'll need to place the template in your project at /src/main/resources/templates/home.html. Let's create that template now.

3.2 Defining the view

In the interest of keeping your home page simple, it should do nothing more than welcome users to the site. The next listing shows the basic Thymeleaf template that defines the Taco Cloud home page.

Listing 1.5 The Taco Cloud home page template

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:th="http://www.thymeleaf.org">
 <head>
  <title>Taco Cloud</title>
 </head>

 <body>
  <h1>Welcome to...</h1>
  <img th:src="@{/images/TacoCloud.png}"/>
 </body>
</html>
```

There's not much to discuss with regard to this template. The only notable line of code is the one with the `<img>` tag to display the Taco Cloud logo. It uses a Thymeleaf `th:src` attribute and an `@{...}` expression to reference the image with a context-relative path. Aside from that, it's not much more than a Hello World page.

Let's talk about that image a bit more. I'll leave it up to you to define a Taco Cloud logo that you like. But you'll need to make sure you place it at the right place within the project.

The image is referenced with the context-relative path /images/TacoCloud.png. As you'll recall from our review of the project structure, static content, such as images, is kept in the /src/main/resources/static folder. That means that the Taco Cloud logo image must also reside within the project at /src/main/resources/static/ images/TacoCloud.png.

Now that you've got a controller to handle requests for the home page and a view template to render the home page, you're almost ready to fire up the application and see it in action. But first, let's see how you can write a test against the controller.

3.3 Testing the controller

Testing web applications can be tricky when making assertions against the content of an HTML page. Fortunately, Spring comes with some powerful test support that makes testing a web application easy.

For the purposes of the home page, you'll write a test that's comparable in complexity to the home page itself. Your test will perform an HTTP `GET` request for the root path / and expect a successful result where the view name is `home` and the resulting content contains the phrase "Welcome to...." The following code should do the trick.

Listing 1.6 A test for the home page controller

```
package tacos;

import static org.hamcrest.Matchers.containsString;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.view;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(HomeController.class)                    (1)
public class HomeControllerTest {

  @Autowired
  private MockMvc mockMvc;                           (2)

  @Test
  public void testHomePage() throws Exception {
    mockMvc.perform(get("/"))                        (3)
      .andExpect(status().isOk())                    (4)
      .andExpect(view().name("home"))                (5)
      .andExpect(content().string(                   (6)
        containsString("Welcome to...")));
  }
```

```
}
```

❶ Web test for HomeController

❷ Injects MockMvc

❸ Performs GET /

❹ Expects HTTP 200

❺ Expects home view

❻ Expects Welcome to...

The first thing you might notice about this test is that it differs slightly from the `TacoCloudApplicationTests` class with regard to the annotations applied to it. Instead of `@SpringBootTest` markup, `HomeControllerTest` is annotated with `@WebMvcTest`. This is a special test annotation provided by Spring Boot that arranges for the test to run in the context of a Spring MVC application. More specifically, in this case, it arranges for `HomeController` to be registered in Spring MVC so that you can send requests to it.

`@WebMvcTest` also sets up Spring support for testing Spring MVC. Although it could be made to start a server, mocking the mechanics of Spring MVC is sufficient for your purposes. The test class is injected with a `MockMvc` object (thanks to the @Autowired annotation) for the test to drive the mockup.

The `testHomePage()` method defines the test you want to perform against the home page. It starts with the `MockMvc` object to perform an HTTP `GET` request for / (the root path). From that request, it sets the following expectations:

- The response should have an HTTP 200 (OK) status.
- The view should have a logical name of `home`.
- The rendered view should contain the text “Welcome to....”

You can run the test in your IDE of choice or with Maven like this:

```
$ mvnw test
```

If, after the `MockMvc` object performs the request, any of those expectations aren't met, then the test will fail. But your controller and view template are written to satisfy those expectations, so the test should pass with flying colors—or at least with some shade of green indicating a passing test.

The controller has been written, the view template created, and you have a passing test. It seems that you've implemented the home page successfully. But even though the test passes, there's something slightly more satisfying with seeing the results in a browser. After all, that's how Taco Cloud customers are going to see it. Let's build the application and run it.

3.4 Building and running the application

Just as we have several ways to initialize a Spring application, we also have several ways to run one. If you like, you can flip over to the appendix to read about some of the more common ways to run a Spring Boot application.

Because you chose to use Spring Tool Suite to initialize and work on the project, you have a handy feature called the Spring Boot Dashboard available to help you run your application inside the IDE. The Spring Boot Dashboard appears as a tab, typically near the bottom left of the IDE window. Figure 1.7 shows an annotated screenshot of the Spring Boot Dashboard.

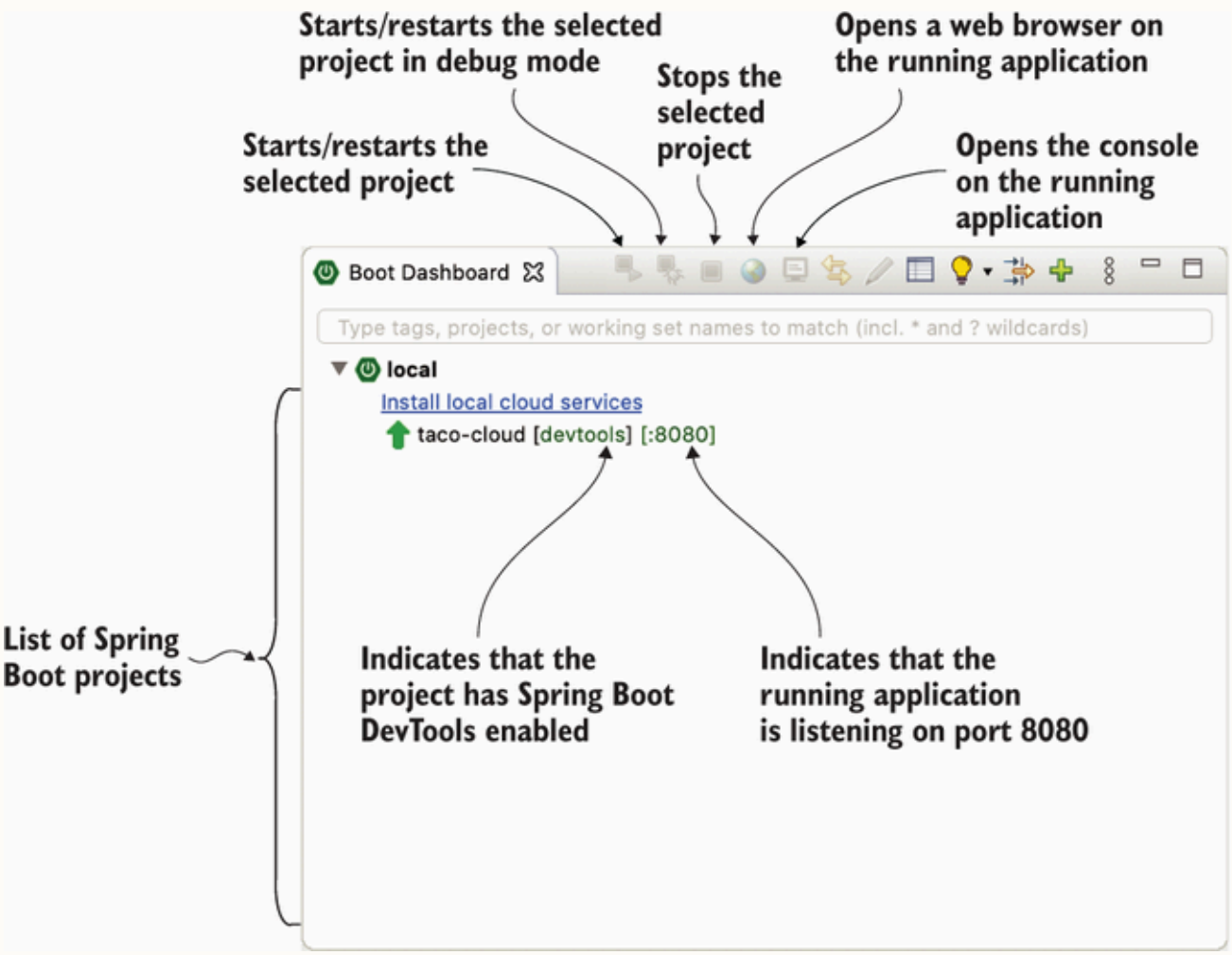


Figure 1.7 Highlights of the Spring Boot Dashboard

I don't want to spend much time going over everything the Spring Boot Dashboard does, although figure 1.7 covers some of the most useful details. The important thing to know right now is how to use it to run the Taco Cloud application. Make sure `taco-cloud` application is highlighted in the list of projects (it's the only application shown in figure 1.7), and then click the start button (the left-most button with both a green triangle and a red square). The application should start right up.

As the application starts, you'll see some Spring ASCII art fly by in the console, followed by some log entries describing the steps as the application starts. Before the logging stops, you'll see a log entry saying Tomcat started on port(s): 8080 (http), which means that you're ready to point your web browser at the home page to see the fruits of your labor.

Wait a minute. Tomcat started? When did you deploy the application to a Tomcat web server?

Spring Boot applications tend to bring everything they need with them and don't need to be deployed to some application server. You never deployed your application to Tomcat

—Tomcat is a part of your application! (I'll describe the details of how Tomcat became part of your application in section 1.3.6.)

Now that the application has started, point your web browser to http://localhost:8080 (or click the globe button in the Spring Boot Dashboard) and you should see something like figure 1.8. Your results may be different if you designed your own logo image, but it shouldn't vary much from what you see in figure 1.8.

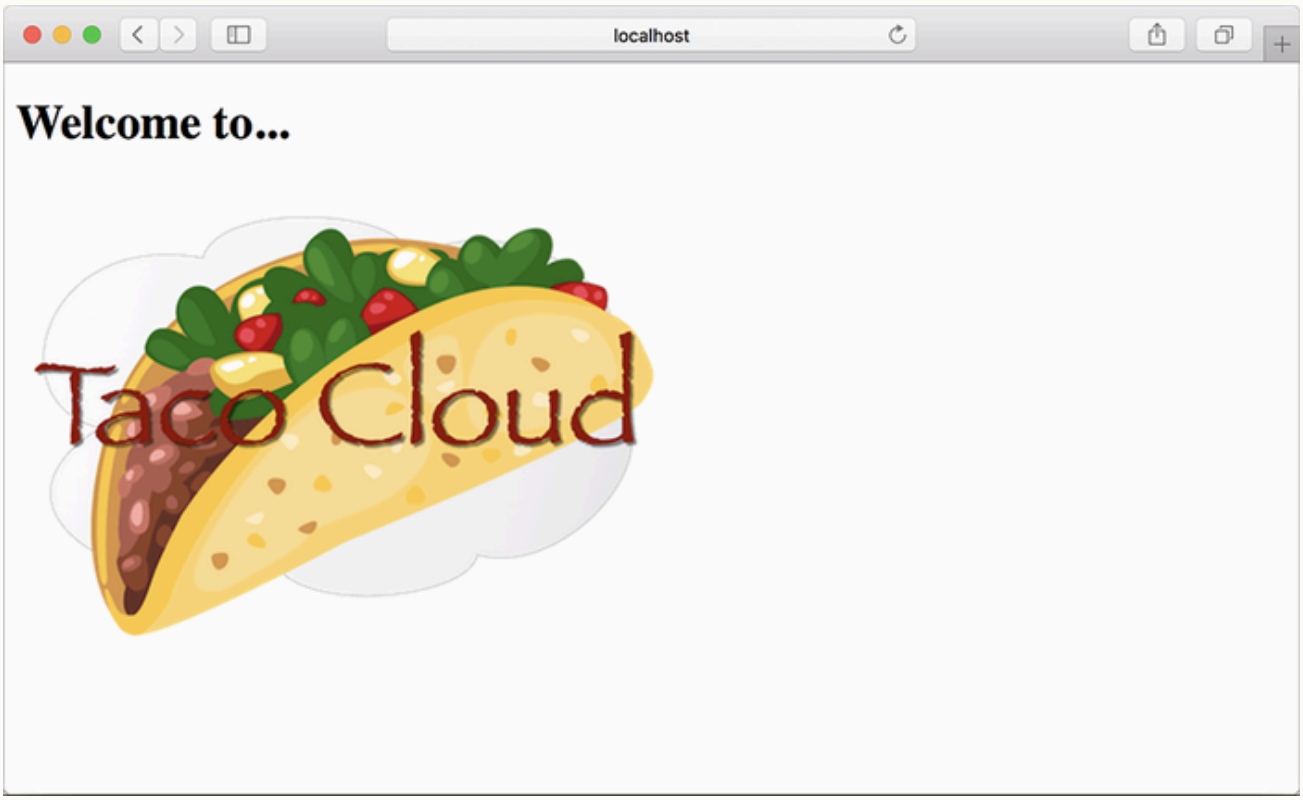


Figure 1.8 The Taco Cloud home page

It may not be much to look at. But this isn't exactly a book on graphic design. The humble appearance of the home page is more than sufficient for now. And it provides you a solid start on getting to know Spring.

One thing I've glossed over up until now is DevTools. You selected it as a dependency when initializing your project. It appears as a dependency in the generated pom.xml file. And the Spring Boot Dashboard even shows that the project has DevTools enabled. But what is DevTools, and what does it do for you? Let's take a quick survey of a couple of DevTools's most useful features.

3.5 Getting to know Spring Boot DevTools

As its name suggests, DevTools provides Spring developers with some handy development-time tools. Among those are the following:

- Automatic application restart when code changes
- Automatic browser refresh when browser-destined resources (such as templates, JavaScript, stylesheets, and so on) change
- Automatic disabling of template caches
- Built in H2 Console, if the H2 database is in use

It's important to understand that DevTools isn't an IDE plugin, nor does it require that you use a specific IDE. It works equally well in Spring Tool Suite, IntelliJ IDEA, and NetBeans. Furthermore, because it's intended only for development purposes, it's smart enough to disable itself when deploying in a production setting. We'll discuss how it does this when you get around to deploying your application in chapter 18. For now, let's focus on the most useful features of Spring Boot DevTools, starting with automatic application restart.

AUTOMATIC APPLICATION RESTART

With DevTools as part of your project, you'll be able to make changes to Java code and properties files in the project and see those changes applied after a brief moment. DevTools monitors for changes, and when it sees something has changed, it automatically restarts the application.

More precisely, when DevTools is active, the application is loaded into two separate class loaders in the Java virtual machine (JVM). One class loader is loaded with your Java code, property files, and pretty much anything that's in the src/main/ path of the project. These are items that are likely to change frequently. The other class loader is loaded with dependency libraries, which aren't likely to change as often.

When a change is detected, DevTools reloads only the class loader containing your project code and restarts the Spring application context but leaves the other class loader and the JVM intact. Although subtle, this strategy affords a small reduction in the time it takes to start the application.

The downside of this strategy is that changes to dependencies won't be available in automatic restarts. That's because the class loader containing dependency libraries isn't automatically reloaded. Any time you add, change, or remove a dependency in your build specification, you'll need to do a hard restart of the application for those changes to take effect.

AUTOMATIC BROWSER REFRESH AND TEMPLATE CACHE DISABLE

By default, template options such as Thymeleaf and FreeMarker are configured to cache the results of template parsing so that templates don't need to be reparsed with every request they serve. This is great in production, because it buys a bit of a performance benefit.

Cached templates, however, are not so great at development time. They make it impossible to make changes to the templates while the application is running and see the results after refreshing the browser. Even if you've made changes, the cached template will still be in use until you restart the application.

DevTools addresses this issue by automatically disabling all template caching. Make as many changes as you want to your templates and know that you're only a browser refresh away from seeing the results.

But if you're like me, you don't even want to be burdened with the effort of clicking the browser's refresh button. It'd be much nicer if you could make the changes and witness the results in the browser immediately. Fortunately, DevTools has something special for those of us who are too lazy to click a refresh button.

DevTools automatically enables a LiveReload server (**http://livereload.com/**) along with your application. By itself, the LiveReload server isn't very useful. But when coupled with a corresponding LiveReload browser plugin, it causes your browser to automatically refresh when changes are made to templates, images, stylesheets, JavaScript, and so on—in fact, almost anything that ends up being served to your browser.

LiveReload has browser plugins for Google Chrome, Safari, and Firefox browsers. (Sorry, Internet Explorer and Edge fans.) Visit **http://livereload.com/extensions/** to find information on how to install LiveReload for your browser.

BUILT-IN H2 CONSOLE

Although your project doesn't yet use a database, that will change in chapter 3. If you choose to use the H2 database for development, DevTools will also automatically enable an H2 console that you can access from your web browser. You only need to point your web browser to http:/ /localhost:8080/h2-console to gain insight into the data your application is working with.

At this point, you've written a complete, albeit simple, Spring application. You'll expand on it throughout the course of the book. But now is a good time to step back and review

what you've accomplished and how Spring played a part.

3.6 Let's review

Think back on how you got to this point. In short, you've taken the following steps to build your Taco Cloud Spring application:

- You created an initial project structure using the Spring Initializr.
- You wrote a controller class to handle the home page request.
- You defined a view template to render the home page.
- You wrote a simple test class to prove your work.

Seems pretty straightforward, doesn't it? With the exception of the first step to bootstrap the project, each action you've taken has been keenly focused on achieving the goal of producing a home page.

In fact, almost every line of code you've written is aimed toward that goal. Not counting Java `import` statements, I count only two lines of code in your controller class and no lines in the view template that are Spring-specific. And although the bulk of the test class utilizes Spring testing support, it seems a little less invasive in the context of a test.

That's an important benefit of developing with Spring. You can focus on the code that meets the requirements of an application, rather than on satisfying the demands of a framework. Although you'll no doubt need to write some framework-specific code from time to time, it'll usually be only a small fraction of your codebase. As I said before, Spring (with Spring Boot) can be considered the *frameworkless framework*.

How does this even work? What is Spring doing behind the scenes to make sure your application needs are met? To understand what Spring is doing, let's start by looking at the build specification.

In the pom.xml file, you declared a dependency on the `Web` and `Thymeleaf` starters. These two dependencies transitively brought in a handful of other dependencies, including the following:

- Spring's MVC framework
- Embedded Tomcat
- Thymeleaf and the Thymeleaf layout dialect

It also brought Spring Boot's autoconfiguration library along for the ride. When the application starts, Spring Boot autoconfiguration detects those libraries and automatically performs the following tasks:

- Configures the beans in the Spring application context to enable Spring MVC
- Configures the embedded Tomcat server in the Spring application context
- Configures a Thymeleaf view resolver for rendering Spring MVC views with Thymeleaf templates

In short, autoconfiguration does all the grunt work, leaving you to focus on writing code that implements your application functionality. That's a pretty sweet arrangement, if you ask me!

Your Spring journey has just begun. The Taco Cloud application only touched on a small portion of what Spring has to offer. Before you take your next step, let's survey the Spring landscape and see what landmarks you'll encounter on your journey.

1.4 Surveying the Spring landscape

To get an idea of the Spring landscape, look no further than the enormous list of checkboxes on the full version of the Spring Initializr web form. It lists over 100 dependency choices, so I won't try to list them all here or to provide a screenshot. But I encourage you to take a look. In the meantime, I'll mention a few of the highlights.

4.1 The core Spring Framework

As you might expect, the core Spring Framework is the foundation of everything else in the Spring universe. It provides the core container and dependency injection framework. But it also provides a few other essential features.

Among these is Spring MVC, Spring's web framework. You've already seen how to use Spring MVC to write a controller class to handle web requests. What you've not yet seen, however, is that Spring MVC can also be used to create REST APIs that produce non-HTML output. We're going to dig more into Spring MVC in chapter 2 and then take another look at how to use it to create REST APIs in chapter 7.

The core Spring Framework also offers some elemental data persistence support, specifically, template-based JDBC support. You'll see how to use `JdbcTemplate` in chapter 3.

Spring includes support for reactive-style programming, including a new reactive web framework called Spring WebFlux that borrows heavily from Spring MVC. You'll look at

Spring's reactive programming model in part 3 and Spring WebFlux specifically in chapter 12.

4.2 Spring Boot

We've already seen many of the benefits of Spring Boot, including starter dependencies and autoconfiguration. Be certain that we'll use as much of Spring Boot as possible throughout this book and avoid any form of explicit configuration, unless it's absolutely necessary. But in addition to starter dependencies and autoconfiguration, Spring Boot also offers the following other useful features:

- The Actuator provides runtime insight into the inner workings of an application, including metrics, thread dump information, application health, and environment properties available to the application.
- Flexible specification of environment properties.
- Additional testing support on top of the testing assistance found in the core framework.

What's more, Spring Boot offers an alternative programming model based on Groovy scripts that's called the Spring Boot CLI (command-line interface). With the Spring Boot CLI, you can write entire applications as a collection of Groovy scripts and run them from the command line. We won't spend much time with the Spring Boot CLI, but we'll touch on it on occasion when it fits our needs.

Spring Boot has become such an integral part of Spring development that I can't imagine developing a Spring application without it. Consequently, this book takes a Spring Boot-centric view, and you might catch me using the word *Spring* when I'm referring to something that Spring Boot is doing.

4.3 Spring Data

Although the core Spring Framework comes with basic data persistence support, Spring Data provides something quite amazing: the ability to define your application's data repositories as simple Java interfaces, using a naming convention when defining methods to drive how data is stored and retrieved.

What's more, Spring Data is capable of working with several different kinds of databases, including relational (via JDBC or JPA), document (Mongo), graph (Neo4j), and others. You'll use Spring Data to help create repositories for the Taco Cloud application in chapter 3.

4.4 Spring Security

Application security has always been an important topic, and it seems to become more important every day. Fortunately, Spring has a robust security framework in Spring Security.

Spring Security addresses a broad range of application security needs, including authentication, authorization, and API security. Although the scope of Spring Security is too large to be properly covered in this book, we'll touch on some of the most common use cases in chapters 5 and 12.

4.5 Spring Integration and Spring Batch

At some point, most applications will need to integrate with other applications or even with other components of the same application. Several patterns of application integration have emerged to address these needs. Spring Integration and Spring Batch provide the implementation of these patterns for Spring applications.

Spring Integration addresses real-time integration where data is processed as it's made available. In contrast, Spring Batch addresses batched integration where data is allowed to collect for a time until some trigger (perhaps a time trigger) signals that it's time for the batch of data to be processed. You'll explore Spring Integration in chapter 10.

4.6 Spring Cloud

The application development world is entering a new era where we'll no longer develop our applications as single-deployment, unit monoliths and will instead compose applications from several individual deployment units known as *microservices*.

Microservices are a hot topic, addressing several practical development and runtime concerns. In doing so, however, they bring to fore their own challenges. Those challenges are met head-on by Spring Cloud, a collection of projects for developing cloud-native applications with Spring.

Spring Cloud covers a lot of ground, and it'd be impossible to cover it all in this book. For a complete discussion of Spring Cloud, I suggest taking a look at *Cloud Native Spring in Action* by Thomas Vitale (Manning, 2020, **www.manning.com/books/cloud-native-spring-in-action**).

4.7 Spring Native

A relatively new development in Spring is the Spring Native project. This experimental project enables compilation of Spring Boot projects into native executables using the GraalVM native-image compiler, resulting in images that start significantly faster and have a lighter footprint.

For more information on Spring Native, see **https://github.com/spring-projects-experimental/spring-native**.

Summary

- Spring aims to make developer challenges easy, like creating web applications, working with databases, securing applications, and microservices.
- Spring Boot builds on top of Spring to make Spring even easier with simplified dependency management, automatic configuration, and runtime insights.
- Spring applications can be initialized using the Spring Initializr, which is web-based and supported natively in most Java development environments.
- The components, commonly referred to as beans, in a Spring application context can be declared explicitly with Java or XML, discovered by component scanning, or automatically configured with Spring Boot autoconfigurations.

Previous chapter
‹ Part 1. Foundational Spring

Next chapter
2 Developing web applications ›